

Build 10 Automations Before Your Competitors Wake Up. One person. Eight revenue streams. \$0 orchestration cost.

33

AUTONOMOUS AGENTS

8

REVENUE STREAMS

292

SCRIPTS BUILT

2hr

DAILY INVOLVEMENT

One Person. Competing Against Ten.

Most solopreneurs spend 30+ hours a week on tasks a Python script can do in 3 minutes. Lead research. Competitor monitoring. Content scheduling. Email personalization. Invoice tracking.

This guide documents the exact system one solopreneur built to run 8 revenue streams with 2 hours of daily involvement. 33 autonomous agents. 292 scripts. \$0/month in orchestration costs. No SaaS. No vendor lock-in.

The core insight: **filesystem = memory, context = disposable scratchpad**. Each agent reads state from files, does one job, writes results, and exits. The next iteration starts fresh. No context crashes. No degradation. Unlimited runtime.

What you get: The complete solopreneur agent architecture, 8 agent configs purpose-built for one-person businesses, Ralph loop patterns for overnight execution, a cold email pipeline that needs no Apollo subscription, and a content machine that produces 90+ pieces per month from operations work.

The 8 Agents a Solopreneur Needs

Lead Finder

Scrapes Google Maps, LinkedIn, and Apollo alternatives. Scores prospects by revenue potential. Outputs a scored CSV by morning. You wake up to 50 pre-qualified leads.

RUNS OVERNIGHT

CRON 2AM

Cold Email Drafter

Reads scored leads. Pulls one specific problem from each prospect's website. Drafts a sub-100-word email using the 6-question framework. Saves drafts to EMAIL/drafts/ for your morning review.

RUNS OVERNIGHT

AI-ASSISTED

Content Repurposer

Takes any piece of work you did today (a build session, a client call, a finding) and outputs 5 content pieces: thread, LinkedIn post, newsletter section, short-form script, community post.

RUNS AFTER SESSIONS

Invoice Tracker

Monitors your invoicing tool via API or CSV export. Flags overdue invoices. Generates follow-up emails. Logs payment status. Zero manual bookkeeping for status checks.

CRON DAILY 9AM

Competitor Monitor

Checks competitor sites, pricing pages, job postings, and changelog pages daily. Writes a morning digest to INTEL/competitor_digest.md. Free. Runs on cron. No paid tools.

CRON 6AM

FREE

SEO Page Generator

Finds low-competition keyword clusters. Generates SEO-optimized landing pages from templates. Deploys to your site automatically. Targets informational queries your competitors aren't covering.

CRON WEEKLY

Digital Product Builder

Monitors what's selling on Gumroad, Whop, and Lemon Squeezy. Identifies gaps. Drafts product outlines. Generates listing copy. The product-to-market cycle goes from days to hours.

AI-ASSISTED

RUNS ON TRIGGER

Revenue Tracker

Polls Stripe, Gumroad, and affiliate dashboards. Aggregates into a single daily summary. Calculates MRR, best-performing products, and revenue per hour of work. Written to FINANCIALS/daily.md.

CRON 8AM

AUTOMATED

System Output (Real Numbers)

Metric	Before Automation	After Automation
--------	-------------------	------------------

Leads researched per week	30-50 (4 hours manual)	200+ (agent runs overnight)
Cold emails personalized per week	20-30 (3 hours)	100+ (reviewed in 20 min)
Competitor intel updates	Weekly (2 hours)	Daily (5 min to read digest)
Content pieces per month	15-20 (scattered)	90+ (from operations work)
Revenue tracking	Weekly spreadsheet (1 hour)	Daily automated summary
Human hours required daily	8-10 hours	2 hours (review + strategy)

The 3-Layer Solopreneur Stack

Three layers, each doing different work on a different schedule. The combination is what produces the leverage.

Layer 1: Always-On (cron + launchd)

Scripts that run 24/7 with zero human involvement. Scraping, monitoring, lead scoring, backups, health checks. This layer never sleeps, never gets distracted, never forgets.

CONTINUOUS

\$0/MONTH

Layer 2: AI-Assisted Sessions (Claude Code)

The 2 hours you spend daily. Strategy decisions, reviewing agent output, building new automations, client work that actually needs your brain. Claude Code helps write the scripts that feed Layer 1.

2 HOURS/DAY

HUMAN IN LOOP

Layer 3: Overnight Ralph Loops

Long-running Claude Code sessions that execute while you sleep. Lead processing, content drafting, research, email sequencing. You launch the loop before bed, wake up to completed work.

RUNS OVERNIGHT

FILESYSTEM MEMORY

Layer 1: 15+ Cron Scripts (Always Running)

Script	Schedule	What It Does
lead_scraper.py	2 AM daily	Google Maps + LinkedIn scrape, outputs raw_leads.csv
lead_scorer.py	3 AM daily	Reads raw leads, scores by fit, saves scored_leads.csv
competitor_watcher.py	6 AM daily	Checks 10 competitor sites for changes, writes digest
content_scheduler.py	7 AM daily	Publishes queued content, updates schedule CSV
revenue_pull.py	8 AM daily	Polls Stripe + Gumroad APIs, writes daily summary
invoice_checker.py	9 AM daily	Flags overdue invoices, generates follow-up drafts
alpha_scraper.py	Every 6h	Scrapes Reddit/Twitter for relevant alpha, saves to staging
health_check.py	Every 2h	Verifies all agents are running, alerts if dead
backup.sh	9 PM daily	Git commit all state files, push to remote

seo_page_gen.py	Sunday 4 AM	Identifies keyword gaps, generates 3-5 landing pages
product_monitor.py	Every 12h	Scans Gumroad/Whop bestsellers, logs gaps
inbox_warmup.py	8 AM weekdays	Sends warmup emails from secondary inboxes
kpi_rollup.py	Sunday 8 AM	Weekly KPI digest across all revenue streams
link_checker.py	Daily 5 AM	Verifies deployed sites are live, alerts on 404s
memory Consolidate.py	11 PM daily	Extracts key learnings from today's agent outputs

Tool cost breakdown for Layer 1: cron (macOS, \$0), launchd (macOS, \$0), Python 3 (\$0), git (\$0), Claude Code (subscription, already paying).
Total overhead for the entire agent layer: \$0/month. The only cost is the Claude Code subscription you already have.

Basic Cron Entry Format

```
# Format: min hour day month weekday command
# Add with: crontab -e

# Lead scraper - runs at 2 AM every day
0 2 * * * cd /path/to/project && python3 agents/lead_scraper.py >> logs/lead_scrape

# Competitor watcher - runs at 6 AM
0 6 * * * cd /path/to/project && python3 agents/competitor_watcher.py >> logs/comp.

# Revenue pull - runs at 8 AM weekdays only
0 8 * * 1-5 cd /path/to/project && python3 agents/revenue_pull.py >> logs/revenue.1

# Weekly SEO page gen - Sunday 4 AM
0 4 * * 0 cd /path/to/project && python3 agents/seo_page_gen.py >> logs/seo.log 2>&
```

```
# Guardian git commit – every hour
0 * * * * cd /path/to/project && git add -A && git commit -m "Auto-save $(date +%F\
```

Layer 2: What the 2 Hours Actually Looks Like

Morning routine (45 min):

- Read revenue summary and competitor digest (10 min)
- Review and send approved cold email drafts (15 min)
- Review content queue, approve or edit (10 min)
- Check lead scores, add notes on top prospects (10 min)

Build session (75 min):

- One focused build task with Claude Code (new agent, new feature, client deliverable)
- Generate content from the session before closing (15 min via content machine)
- Launch overnight loop if needed, set PROMPT.md task list

The Overnight Loop (Ralph Pattern)

While you sleep, your agents work. This is the pattern that makes it possible.

The Command

```
while ;; do
  cat PROMPT.md | claude --dangerously-skip-permissions --print
done
```

That is the entire loop. The sophistication is in how PROMPT.md is structured and what state files exist on disk.

Why this works: Each iteration gets a fresh 200K context window. No degradation. No crashes. The filesystem is the memory. Git is the version control. The bash loop is the scheduler. Every iteration reads

state from files, does exactly one task, writes results back, and exits cleanly.

PROMPT.md Structure for Solopreneurs

Role

You are an autonomous solopreneur agent. Working directory: /path/to/project

State (read every iteration)

Read .ralph/progress.md for what you completed last iteration.

Read .ralph/state.json for current metrics and counts.

Task List (pick highest priority, do ONE, then EXIT)

1. If LEADS/scored_leads.csv has rows with status=UNPROCESSED:
 - Take the top 20 by score
 - Research each company website (fetch + parse)
 - Find one specific problem visible on their site
 - Draft personalized cold email using EMAIL/templates/framework.md
 - Save draft to EMAIL/drafts/{company_slug}.txt
 - Mark rows as PROCESSED in scored_leads.csv
2. If EMAIL/drafts/ has files not yet reviewed:
 - Check each draft for quality (specific problem, under 100 words, clear ask)
 - Mark as APPROVED or NEEDS_EDIT with notes
 - APPROVED drafts move to EMAIL/ready_to_send/
3. If CONTENT/raw_sessions/ has new files from today:
 - Read each session note
 - Generate 3 tweets + 1 thread from the best insight
 - Save to CONTENT/queue/ as PENDING_REVIEW
4. If nothing pending:
 - Read INTEL/competitor_digest.md
 - Extract top 3 insights worth sharing as content
 - Generate 2 tweets per insight
 - Save to CONTENT/queue/

Completion Criteria

Task 1 done: EMAIL/drafts/ has N new .txt files + scored_leads.csv updated

Task 2 done: All drafts in EMAIL/drafts/ have a status marker

Task 3 done: CONTENT/queue/ has new files with today's date prefix

Task 4 done: CONTENT/queue/ has 6 new tweet files

Rules

- ONE task per iteration, then EXIT immediately
- Max runtime: 10 minutes per iteration
- Write completion note to .ralph/progress.md
- Update .ralph/state.json with counts
- If blocked (API error, missing file), write to .ralph/blockers.md and move to next iteration
- NEVER loop within a single iteration

Example: Overnight Lead Processing Loop

ROLE: autonomous solopreneur agent

TASK: Review LEADS/new_prospects.csv, score each lead by revenue potential (company size, tech stack, visible problems on site), generate cold email draft for top 20, save to EMAIL/drafts/

DONE WHEN:

- LEADS/scored.csv exists with all rows scored
- EMAIL/drafts/ has 20 .txt files
- .ralph/progress.md updated with count and top lead

READ FIRST: PLAYBOOKS/cold_email_framework.md

STATE FILE: .ralph/state.json

BLOCKERS: .ralph/blockers.md

What to Schedule Overnight

Good overnight tasks:

- Lead research and scoring (reads sites, no accounts needed)
- Cold email drafting (writes files, no sending)
- Content drafting from session notes (writes files, no posting)
- Competitor research (reads public pages)
- SEO keyword analysis (reads APIs, writes reports)
- Product listing copy generation (writes files)
- Weekly KPI summaries (reads local files, writes digest)

Do NOT run overnight without human review:

- Posting to social media (needs your voice check)
- Sending cold emails (one bad batch damages your domain reputation)

- Making purchases or API calls that cost money
- Billing operations or invoice sending
- Publishing to your main website without review

Launch Before Bed, Read in the Morning

```
# Launch overnight loop in background
nohup bash -c "cd /path/to/project && while :; do \
  cat agents/overnight/PROMPT.md | claude --dangerously-skip-permissions --print; \
  sleep 30; done" > logs/overnight.log 2>&1 &

echo "Loop running. PID: $!"
echo $! > .overnight_pid

# Check progress from another terminal
tail -f logs/overnight.log

# Stop the loop
kill $(cat .overnight_pid)
```

Time Leverage Pyramid

Not everything should be automated. The goal is to free your brain for the work only you can do.

Layer 1: Fully Automate (Never Touch Again)

Mechanical tasks that follow clear rules

Scraping public data. Formatting files. Distributing content to queues. Monitoring live URLs. Lead scoring against criteria you defined. File organization. Log rotation. Backup commits. Health checks. Anything that could be described as an if-then rule.

FULLY AUTOMATED

CRON OR TRIGGER

Layer 2: AI-Assist (Your Review, Agent's Draft)

Creative or contextual work that benefits from drafts

Cold email personalization (agent drafts, you edit or approve). Content from session notes (agent generates, you voice-check). Research synthesis (agent reads 50 sources, you read the digest). Proposal sections (agent drafts, you refine). Product listing copy (agent writes, you tighten).

AI-ASSISTED

REVIEW REQUIRED

Layer 3: Keep Human (Your Competitive Advantage)

Judgment, relationships, and strategic direction

Relationship management (no agent can replace you in a conversation). Pricing decisions. Creative direction and brand voice. Which opportunities to pursue. When to kill something that isn't working. Client calls. Negotiation. Any decision that depends on context an agent cannot read.

HUMANS ONLY

Time Savings Table

Task	Before (manual)	After (automated)	Effort to Automate
Competitor monitoring	2h/week	5 min/day (read digest)	1 Claude Code session
Lead finding	4h/week	0h (agent runs overnight)	2 sessions
Cold email personalization	3h/week	20 min/day (review + send)	1 session
Content scheduling	3h/week	20 min/day (review queue)	1 session

Revenue tracking	1h/week	2 min/day (read summary)	1 session
Invoice follow-ups	1h/week	5 min/week (approve drafts)	1 session
SEO research	4h/week	30 min/week (review targets)	2 sessions
Social media posting	2h/week	15 min/day (review + post)	1 session
Total	~20h/week	~2.5h/week	10 Claude Code sessions

The math: 10 build sessions to set up the stack. Each session is 60-90 minutes. After that, 17+ hours per week freed permanently. The payback period is under two weeks.

What to Build First

Priority order based on time-to-value:

1. Competitor monitor (one Python script, cron entry, free, runs in 20 minutes)
2. Revenue tracker (API calls to processors, 30 minutes to set up)
3. Lead scorer (define your ICP criteria once, scores forever)
4. Cold email drafter (biggest time save per hour of setup)
5. Content repurposer (turns every work session into posts)

The \$0 Content Machine

90+ content pieces per month. Every piece comes from work you were already doing. No separate content creation time needed.

The 3-Tweet Rule

Every build session produces at least 3 tweets + 1 thread. This is non-negotiable. You built something, you solved something, you learned something. That is content. The agent converts it before you close the session.

Implementation: Before ending any Claude Code session, run the content generation prompt (below). Takes 5 minutes. Outputs to CONTENT/queue/ as PENDING_REVIEW. You review during your morning routine. Posts go out on schedule via cron.

Session-to-Content Prompt Template

You just completed a work session. Here is what was built / learned / decided:

[PASTE SESSION SUMMARY OR KEY OUTPUTS HERE]

Generate the following:

1. Three standalone tweets (max 240 chars each):
 - Tweet 1: The specific thing built or discovered (lead with the number or result)
 - Tweet 2: The method or pattern that made it work
 - Tweet 3: The counter-intuitive or non-obvious angle
2. One thread (6-8 tweets):
 - Hook tweet that stops the scroll (problem statement or result)
 - 5-6 body tweets that teach the method step by step
 - CTA tweet at the end (what they should do next)

Rules:

- No em dashes
- No "game-changing," "exciting," "deep dive," "leverage" as a verb
- No generic AI wisdom. Only specific, verifiable, executable information
- Lead with specifics: numbers, tool names, exact steps
- Write for someone who could do this TODAY with what you've told them

Content Pipeline Flow

Work session (build, research, client call)

|

```

v
Session notes --> content generation prompt
|
v
CONTENT/queue/ (PENDING_REVIEW)
|
v
Morning review (5-10 min)
|
+--> APPROVED --> CONTENT/ready_to_post/ --> cron scheduler
|
+--> NEEDS_EDIT --> edit inline --> APPROVED
|
+--> REJECTED --> CONTENT/archive/ (pattern still useful later)

```

Repurposing Matrix

Source	Content Pieces It Generates
One research finding	Twitter thread + LinkedIn post + newsletter section + short-form script + community post = 5 pieces
One build session	3 tweets + 1 thread + 1 "how I built it" post = 5 pieces
One competitor observation	2 tweets + 1 LinkedIn hot take + 1 newsletter breakdown = 4 pieces
One client win	Case study post + 3 tweets on the lesson + testimonial graphic = 5 pieces
One mistake made	2 "what I learned" tweets + 1 thread on what to do instead = 3 pieces

The Anti-Slop Checklist

Before approving any content piece, verify it passes all five:

- No filler words: "exciting," "game-changing," "innovative," "deep dive," "leverage" (as a verb), "unlock," "journey"
- No em dashes used as sentence separators
- At least one specific thing named (tool, number, exact step, company name)

- Could the reader do something with this TODAY? If not, it's too vague
- Would you post this from your personal account without cringing? If not, rewrite

Scheduling Without Buffer

Buffer costs money. You don't need it. CSV import method:

1. Maintain CONTENT/schedule.csv with columns: platform, text, datetime, status
2. cron script reads the CSV every morning at 7 AM
3. Posts anything with status=APPROVED and datetime in the next 24 hours
4. Updates status to POSTED after sending via platform API
5. 30 days of content loaded in 5 minutes via bulk CSV append

```
# content_scheduler.py (simplified core logic)
import csv, datetime, requests

schedule = list(csv.DictReader(open("CONTENT/schedule.csv")))
now = datetime.datetime.now()
window = now + datetime.timedelta(hours=24)

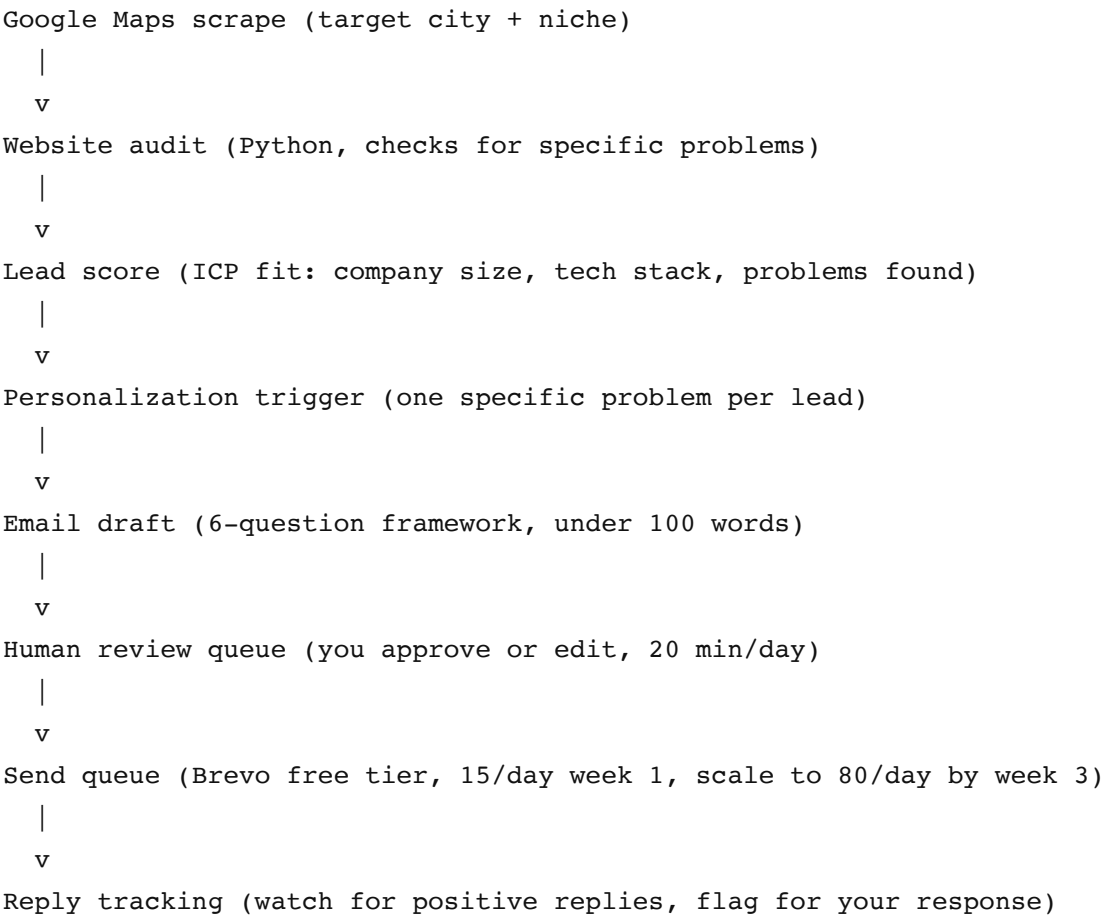
to_post = [
    row for row in schedule
    if row["status"] == "APPROVED"
    and now <= datetime.datetime.fromisoformat(row["datetime"]) <= window
]

for post in to_post:
    # platform-specific API call here
    print(f"Posting to {post['platform']}: {post['text'][:50]}...")
    # update status to POSTED in CSV
```

Cold Email Pipeline

Full pipeline from zero to booked calls. No Apollo. No Hunter. No \$200/month outbound tool.

The Full Pipeline



The 6-Question Email Framework

Question	What It Produces in the Email	Max Words
What do you do?	One sentence on your service	15
Who for?	Name the exact type of company you serve	10
How?	The specific method (not vague)	15
What problem solved?	The exact pain, named directly	15
Proof?	One number or one named outcome	10

ROI / ask?	Specific result they get + one question CTA	15
------------	---	----

Example output using the framework: "I build automated lead pipelines for solo consultants. Most of my clients were spending 4–6 hours a week on manual research before switching. I noticed [SPECIFIC PROBLEM FROM THEIR SITE]. Would a 20-minute call make sense to see if the same approach works for you?" Under 80 words. Personalized. Specific. One ask.

Infrastructure Setup (\$37/month total)

Tool	Cost	What It Does
3 warmed sending domains	\$12/mo (3 x \$4 .com domains)	Separate from your main domain, protects reputation
Google Workspace inboxes (3)	\$18/mo (3 x \$6)	Professional sending addresses on each domain
Brevo (Sendinblue)	\$0 (free tier, 300/day)	Sends your approved drafts, tracks opens and replies
Python scripts	\$0	Scraping, scoring, drafting, monitoring
Total	\$30–37/mo	Full outbound pipeline, 5–10 booked calls/week target

Safe Sending Limits

Week	Emails/Day Per Inbox	Total Daily Volume	Notes
Week 1	5/day	15/day	Warmup period, low volume, high reply rate threshold
Week 2	10/day	30/day	Increase only if open rate > 40% on week 1

Week 3	20-25/day	60-75/day	Full ramp, monitor spam complaints daily
Week 4+	25-30/day	75-90/day	Stable volume, focus on reply rate over volume

Website Audit Script (Core Logic)

```
import requests
from bs4 import BeautifulSoup

def audit_site(url: str) -> dict:
    """Find specific problems on a prospect's site."""
    try:
        resp = requests.get(url, timeout=10, headers={"User-Agent": "Mozilla/5.0"})
        soup = BeautifulSoup(resp.text, "html.parser")
        text = soup.get_text(separator=" ", strip=True).lower()

        problems = []

        # Check for missing trust signals
        if not any(w in text for w in ["testimonial", "review", "case study", "client"])
            problems.append("no social proof visible")

        # Check for vague value prop
        title = soup.title.string if soup.title else ""
        if len(title) < 20 or "welcome" in title.lower():
            problems.append("unclear homepage value prop")

        # Check for missing CTA
        ctas = soup.find_all(["button", "a"], string=lambda t: t and
            any(w in t.lower() for w in ["book", "schedule", "contact", "get started"])
        if len(ctas) < 2:
            problems.append("weak or missing call-to-action")

        # Check for slow load signals (image-heavy)
        images = soup.find_all("img")
        if len(images) > 20:
            problems.append("likely slow load time from unoptimized images")

        return {"url": url, "problems": problems, "top_problem": problems[0] if problems else ""}
```

```
except Exception as e:
    return {"url": url, "problems": [], "error": str(e)}
```

The Personalization Trigger

The audit script finds a real problem on their site. That problem becomes the opening line of the email. This is what separates a 0.5% reply rate from a 5% reply rate.

Pattern: "I noticed [SPECIFIC THING ON THEIR SITE]. That's usually what [your ICP] faces before [the outcome you deliver]." One sentence. Specific. Not generic. The agent finds it. You confirm it makes sense. Done.

Copy-Paste Templates

All templates are ready to use. Swap in your specifics and run.

Template 1: PROMPT.md for Overnight Lead Processing

```
# Role
You are an autonomous lead processing agent for [YOUR BUSINESS].
Working directory: /path/to/project

# State
Read .ralph/progress.md for what was completed last iteration.
Read .ralph/state.json for current metrics.

# Task (pick highest priority, do ONE, EXIT when done)
1. If LEADS/raw_leads.csv has rows with status=UNSCORED:
    - Read ICP criteria from PLAYBOOKS/icp.md
    - Score each lead 1-10 on fit (company size, niche, problems, tech stack)
    - Write scores to LEADS/scored_leads.csv
    - Mark original rows as SCORED

2. If LEADS/scored_leads.csv has rows with score >= 7 and status=UNPROCESSED:
    - Take top 20 by score
    - Fetch each company website
```

- Run audit (find top visible problem)
- Draft email using EMAIL/templates/framework.md
- Save to EMAIL/drafts/{company_slug}.txt
- Mark rows as PROCESSED

3. If EMAIL/drafts/ has files with no status marker:

- Review each for quality (specific, under 100 words, one ask)
- Add APPROVED or NEEDS_EDIT marker at top of file
- Move APPROVED files to EMAIL/ready_to_send/

Completion Criteria

State in .ralph/progress.md what was done and how many records processed.

Update .ralph/state.json: {"leads_scored": N, "drafts_created": N, "drafts_approved": N}

Rules

- ONE task per iteration. EXIT cleanly after completing it.
- Max 10 minutes per iteration.
- If blocked, write to .ralph/blockers.md and skip to next task.

Template 2: Cold Email Framework (Fill In)

Subject: [SPECIFIC OBSERVATION] at [COMPANY NAME]

Hi [FIRST NAME],

[One sentence: what you noticed on their site or about their business]

I [WHAT YOU DO] for [EXACT TYPE OF COMPANY]. Most clients come to me when [SPECIFIC]

[One outcome + one proof point: "Last month I helped [TYPE OF CLIENT] reduce [X] by [Y]"]

Worth a 15-minute call to see if the same approach fits [COMPANY NAME]?

[YOUR NAME]

Template 3: Cron Entry for Daily Competitor Monitoring

competitor_watcher.py – add to crontab with: crontab -e

Runs at 6 AM daily, writes digest to INTEL/competitor_digest.md

0 6 * * * cd /path/to/project && python3 agents/competitor_watcher.py >> logs/competitor_digest.md

```
# competitor_watcher.py core logic:
COMPETITORS = [
    {"name": "CompetitorA", "url": "https://competitora.com", "pricing_url": "https://competitora.com/pricing"},
    {"name": "CompetitorB", "url": "https://competitorb.com", "jobs_url": "https://competitorb.com/jobs"}
]

# For each competitor:
# 1. Fetch homepage, compare to yesterday's snapshot (hash comparison)
# 2. Fetch pricing page, extract prices with regex
# 3. Fetch jobs page, count open roles (hiring signals)
# 4. Write changes to INTEL/competitor_digest.md
# 5. Git commit the digest (creates history)
```

Template 4: Content Generation Prompt

I just completed a work session. Here is what happened:

SESSION TYPE: [build / research / client call / problem-solving]

WHAT WAS BUILT OR LEARNED: [paste your session notes or key output]

KEY RESULT OR FINDING: [the one thing worth sharing]

AUDIENCE: [indie hackers / solopreneurs / [your niche]]

Generate:

TWEETS (3):

Tweet 1 (result-first): Start with the specific outcome or number. Then explain what happened.

Tweet 2 (method): The exact steps or pattern that made it work. Actionable. Max 240 characters.

Tweet 3 (counter-intuitive): The non-obvious angle or common mistake to avoid. Max 240 characters.

THREAD (7 tweets):

1. Hook: problem or result that stops the scroll

2-6. Body: teach the method, step by step, each tweet standalone

7. CTA: what they should do next

RULES:

- No em dashes
- No "exciting," "game-changing," "deep dive," "leverage" as verb
- Specific tool names, numbers, exact steps only
- Each tweet works standalone (no "as I mentioned above")

Template 5: Basic Agent Python Script

```
#!/usr/bin/env python3
"""Template: Autonomous solopreneur agent with state persistence."""
import json
from pathlib import Path
from datetime import datetime

PROJECT_ROOT = Path(__file__).resolve().parent.parent
STATE_FILE = PROJECT_ROOT / "agents" / "my_agent" / "state.json"
LOG_FILE = PROJECT_ROOT / "logs" / "my_agent.log"

def load_state() -> dict:
    if STATE_FILE.exists():
        return json.loads(STATE_FILE.read_text())
    return {"cycles": 0, "last_run": None, "results": []}

def save_state(state: dict) -> None:
    state["last_run"] = datetime.now().isoformat()
    state["cycles"] += 1
    STATE_FILE.write_text(json.dumps(state, indent=2))

def log(msg: str) -> None:
    with open(LOG_FILE, "a") as f:
        f.write(f"[{datetime.now().isoformat()}] {msg}\n")

def main() -> None:
    state = load_state()
    log(f"Cycle {state['cycles'] + 1} starting")

    # 1. Read inputs (CSV, API, scraped data)
    # 2. Process / score / decide
    # 3. Write outputs (files, queues, state)
    # 4. Log result

    result = {"action": "example", "status": "success", "count": 0}
    state["results"].append(result)
    save_state(state)
    log(f"Done: {result}")

if __name__ == "__main__":
    main()
```

Template 6: Weekly Review

```

# Weekly Review Template – run every Sunday

## SHIPPED THIS WEEK
- [What automations went live]
- [What content was published]
- [What leads were sent]
- Revenue: ${X} (from revenue_tracker daily summaries)

## IN QUEUE
- [Drafts waiting for your review]
- [Leads scored but not emailed yet]
- [Content approved but not posted]

## STUCK / BLOCKED
- [Agents that failed this week and why]
- [Tasks that needed human input you haven't provided]
- [Missing API keys or credentials blocking a script]

## KILL LIST
- [Agents producing no useful output – cut them]
- [Content formats getting zero engagement – stop making them]
- [Lead sources with 0% reply – try different scrape target]

## DOUBLE DOWN
- [What worked better than expected – increase frequency]
- [Which content format got the most replies – make more]
- [Which lead source had the highest score-to-call ratio – prioritize]

## NEXT WEEK PRIORITY
1. [One new automation to build]
2. [One existing automation to improve]
3. [One process to fully hand off to an agent]

```

Quick start: Template 5 (basic agent) + Template 1 (overnight PROMPT.md) + Template 3 (cron entry). You will have an autonomous lead processing loop running in under two hours. Add Template 2 and Template 4 to wire in the content machine. The whole stack ships in one weekend.

Built by a solopreneur running 33 agents on a MacBook with 2 hours of daily involvement.

Not theory. A live system.

Works offline. No tracking. No external requests. Open this file in any browser.